

Semester Report — MetaMIRAGE RAG System Development

1. Overview

This semester, I designed and implemented an end-to-end **retrieval-augmented generation (RAG)** system tailored for domain-specific reasoning, particularly in agriculture. The system, referred to as **MetaMIRAGE**, integrates structured metadata, adaptive retrieval strategies, and dynamic knowledge ingestion to improve the reliability and contextual grounding of model outputs.

The work spans four major components:

1. **Offline data ingestion and vector database construction**
2. **Runtime RAG agent with adaptive decision-making**
3. **Batch inference pipeline for scalable generation**
4. **Dynamic ablation framework for controlled experimentation**

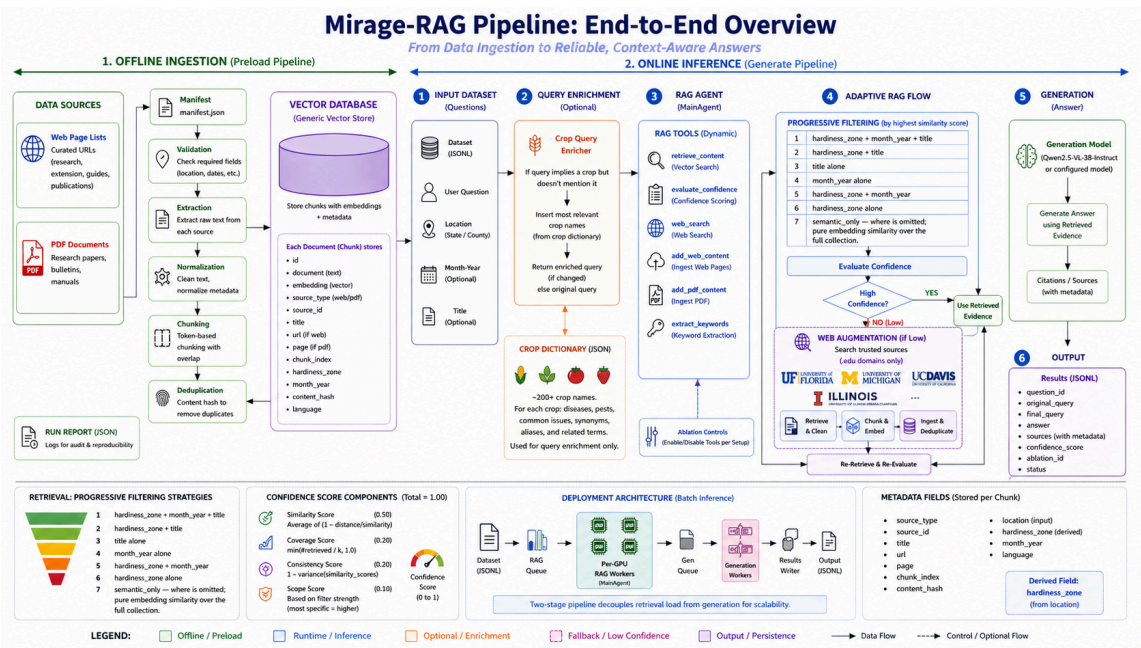


Figure 1 — System overview: offline ingestion, runtime RAG, inference, and experimentation.

2. System Architecture

The system follows a modular pipeline:

- **Offline Stage**
 - Data ingestion (web, PDFs, CSV)
 - Chunking, embedding, and metadata tagging
 - Storage in a persistent Chroma vector database
- **Runtime Stage**

- Query processing and optional enrichment
 - Retrieval with metadata-aware filtering
 - Confidence evaluation
 - Conditional web search and ingestion
- **Inference Stage**
 - Multi-GPU batch processing
 - RAG + generation pipeline
 - JSONL output logging

3. Vector Database and Ingestion Pipeline

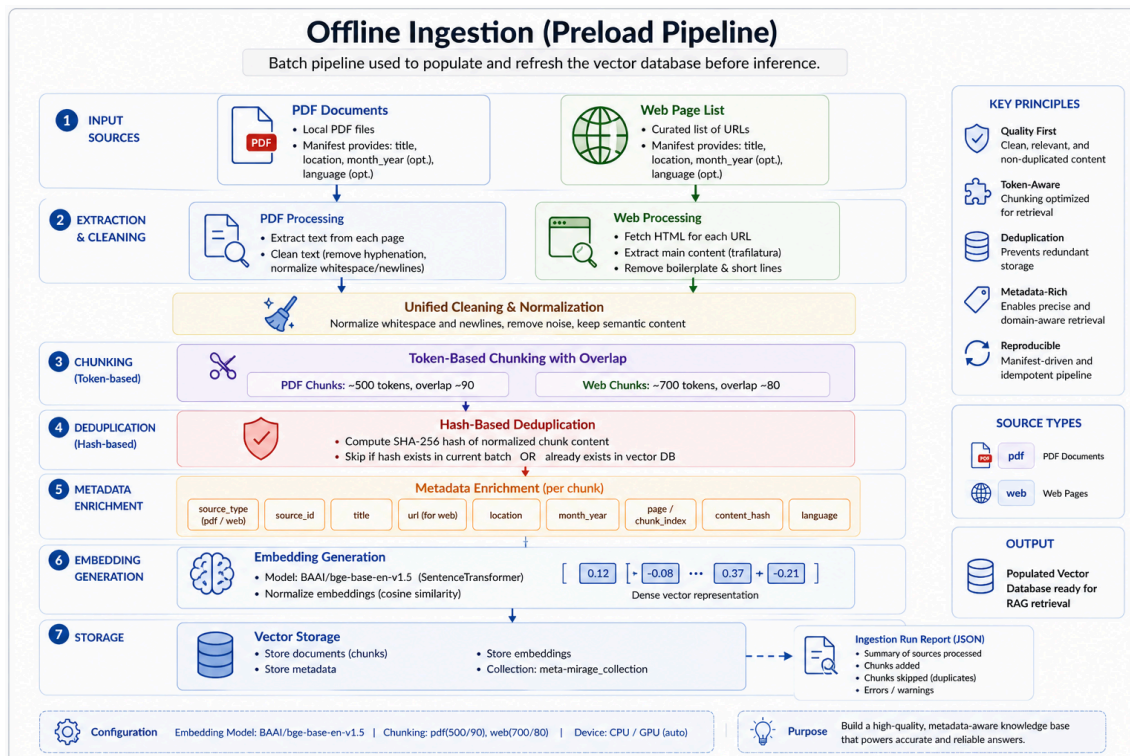


Figure 2 — Offline ingestion pipeline from raw sources to the vector database.

3.1 Database Design

- Implemented a **persistent Chroma vector database**
- Used **sentence-transformer embeddings** for semantic search
- Designed a **canonical metadata schema**, including:
 - location
 - hardness_zone
 - month_year
 - title
 - source_type , source_id , etc.

This metadata enables domain-aware filtering and improves retrieval precision.

3.2 Content Processing

Developed utilities for:

- **Token-based chunking** with overlap
- **Content hashing** for deduplication
- **Normalization and cleaning** for web and PDF text

Ensured ingestion is:

- **Idempotent**
- **Scalable**
- **Consistent across sources**

3.3 Data Sources

Implemented ingestion for:

- Web pages (via structured URL lists)
- PDF documents (page-aware extraction)

All sources are processed through a **manifest-driven preload pipeline**, ensuring reproducibility and auditability.

4. Metadata-Aware Retrieval

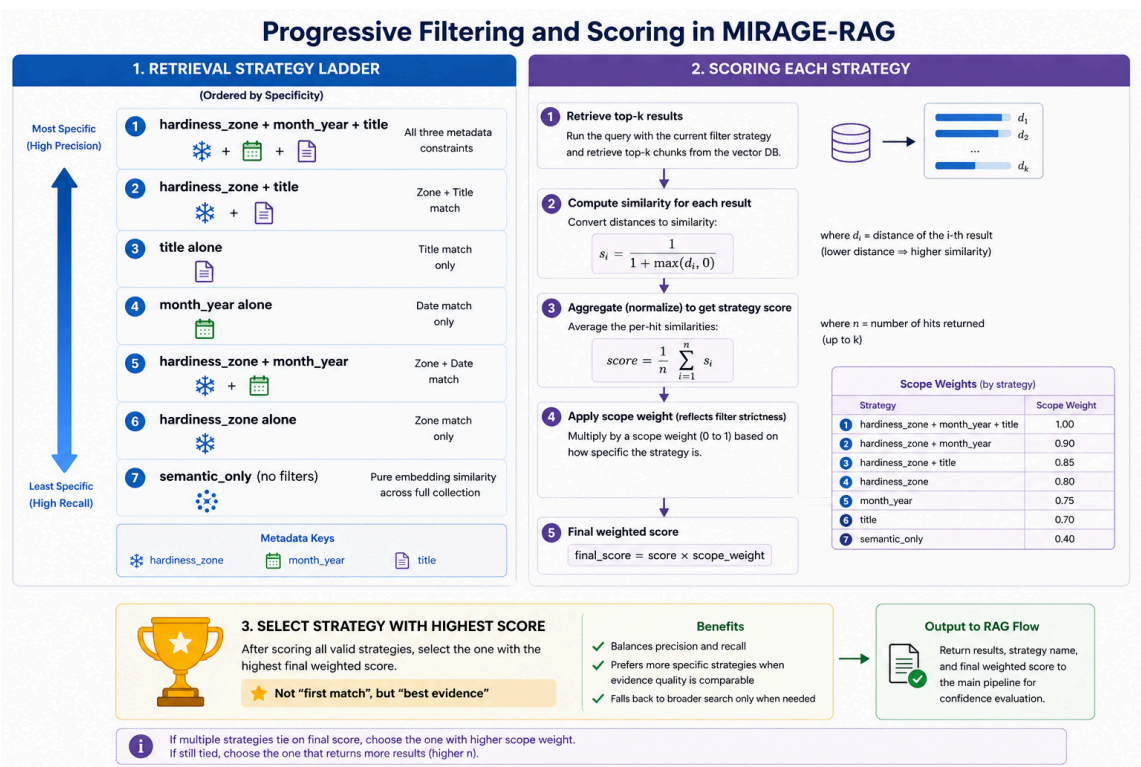


Figure 3 — Progressive filtering strategy across metadata combinations.

4.1 Progressive Filtering

Designed a **priority-based retrieval strategy** that evaluates multiple metadata filters:

- Most specific:
 - location + hardiness_zone + month_year + title
- Intermediate:
 - location + hardiness_zone + month_year
 - location + hardiness_zone
 - location only
- Least specific:
 - semantic-only retrieval

Instead of stopping at the first match, the system:

- Evaluates all candidate strategies
- Computes similarity scores
- Selects the **best-performing strategy**

This ensures robustness when metadata is incomplete or noisy.

4.2 Similarity Scoring

- Converted distances into normalized similarity scores
- Aggregated scores across top-k results
- Selected the best retrieval strategy based on average similarity

5. Confidence Evaluation Framework

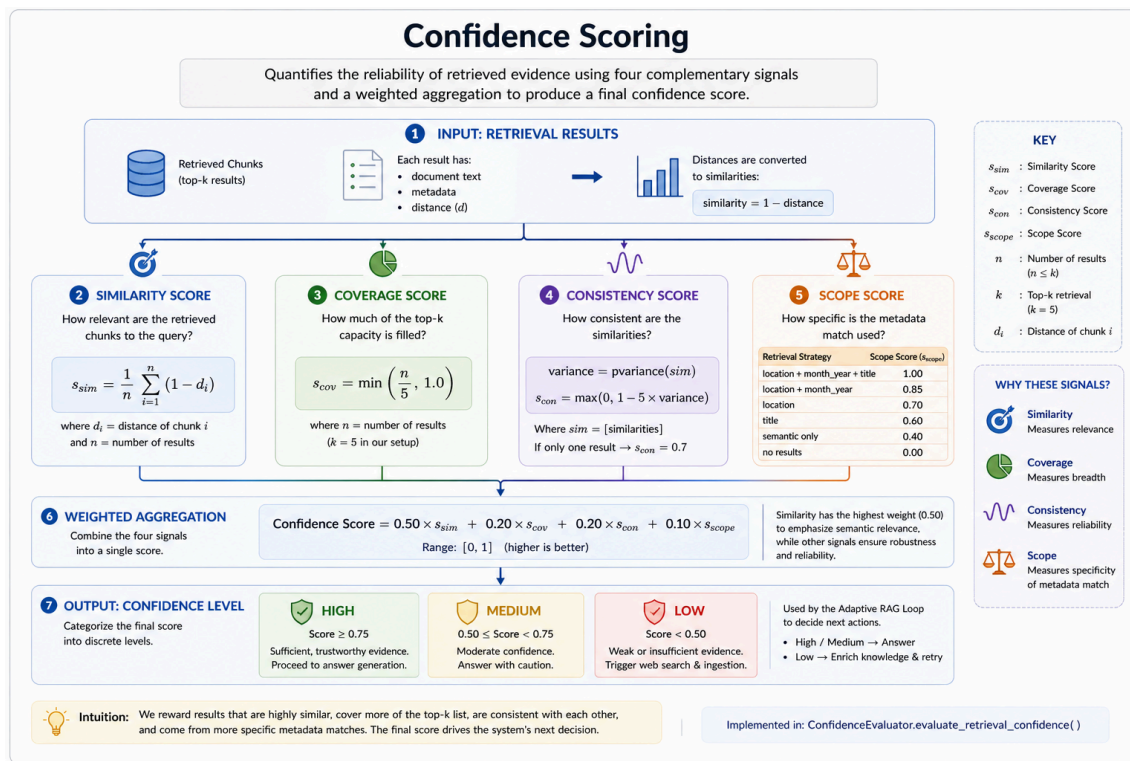


Figure 4 — Confidence evaluation components and flow.

Developed a **multi-factor confidence scoring system** to assess retrieval quality.

Components:

- **Similarity Score** – relevance of retrieved chunks
- **Coverage Score** – number of retrieved results
- **Consistency Score** – agreement across results
- **Scope Score** – strength of metadata filtering used

Output:

- Confidence score (0–1)
- Confidence level: **high / medium / low**

This mechanism enables **adaptive decision-making** in the RAG pipeline.

6. Adaptive RAG Decision Loop

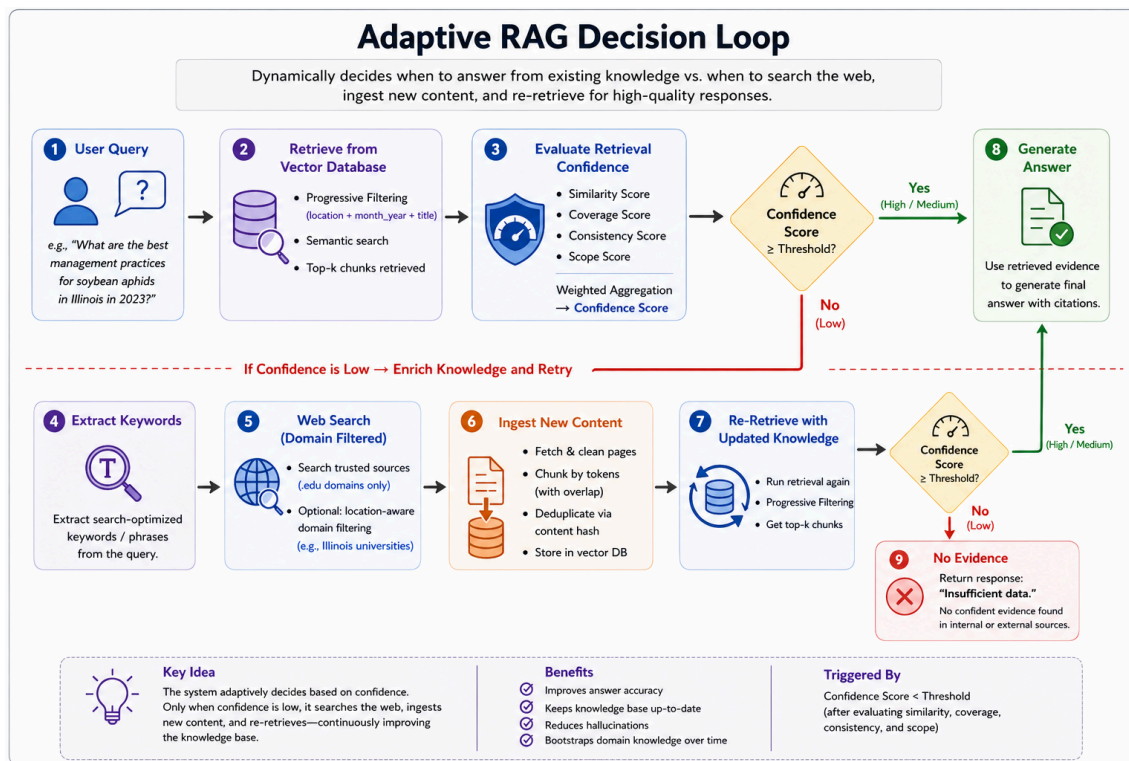


Figure 5 — Runtime decision loop with confidence-gated web augmentation.

Implemented a **dynamic decision loop**:

1. Retrieve from vector database
2. Evaluate confidence
3. If confidence is:
 - **High** → return results
 - **Medium** → return results with caution
 - **Low** → trigger web search and ingestion

Web Augmentation

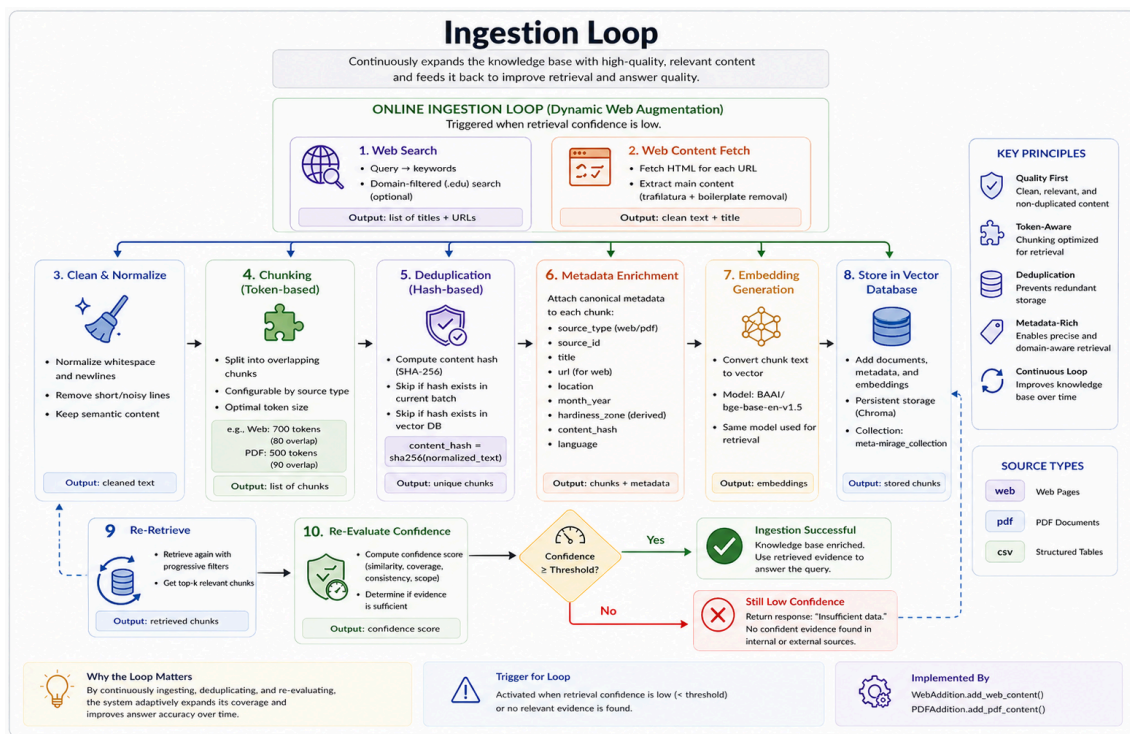


Figure 6 — Web augmentation and ingest-re-retrieve loop.

- Integrated a web search module
- Extracted and cleaned webpage content
- Dynamically ingested new knowledge into the database
- Re-ran retrieval after ingestion

If confidence remains low:

- System returns **"Insufficient data"**

This enables **real-time knowledge expansion**.

7. Query Enrichment (Crop Dictionary)

Developed an optional **query enrichment module**:

- Uses a **crop dictionary JSON**
- Detects missing crop references in queries
- Inserts relevant crop names into the query

Key properties:

- Does not modify the database
- Only enhances query text
- Includes strict fallback to original query on failure

8. Batch Inference Pipeline

8.1 Multi-GPU Design

Implemented a scalable inference system:

- Parallel RAG workers (one per GPU)
- Queue-based request/response handling
- Dedicated generation workers

8.2 Workflow

1. Load dataset
2. Build query (with optional location + enrichment)
3. Run RAG pipeline
4. Append retrieved evidence to query
5. Generate final response
6. Save results in JSONL format

8.3 Fault Handling

- **Soft failures** → fallback to query-only generation
- **Hard failures** → retry mechanism
- Automatic **Chroma rebind** for stale handles

9. Ablation Framework

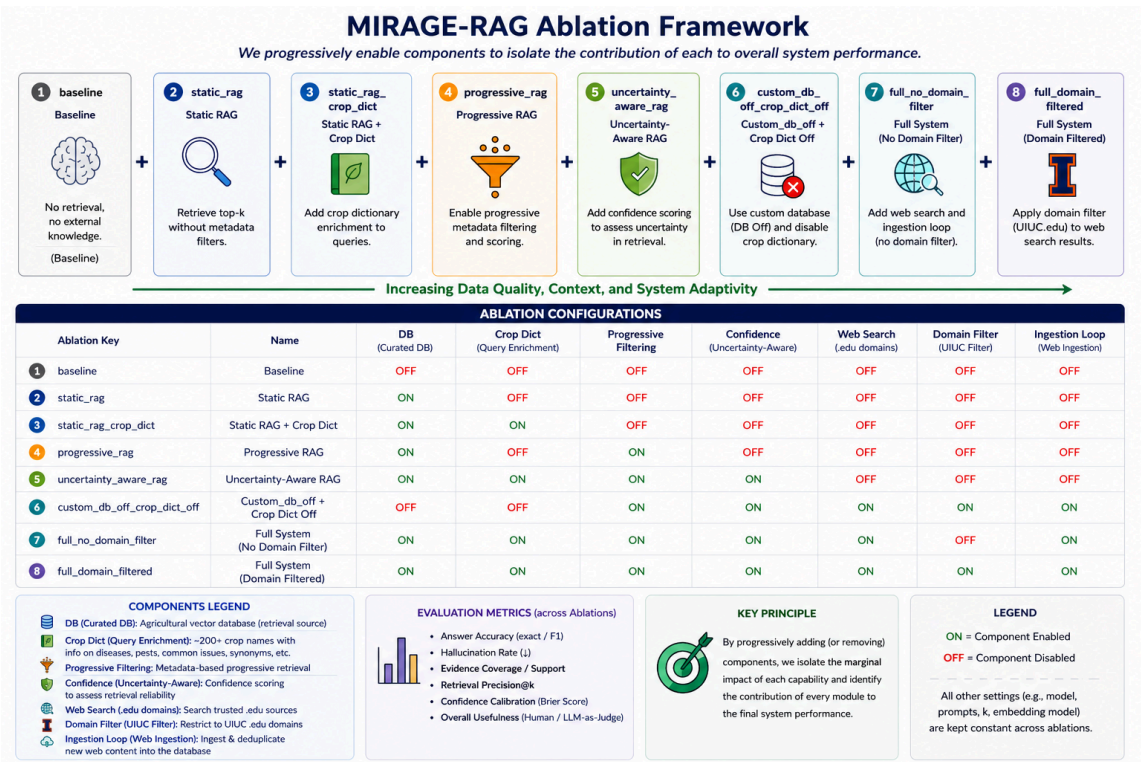


Figure 7 — Ablation framework: centralized config and templated behavior.

The ablation framework evolved into a dynamic experimentation layer that supports fast, reproducible research iterations.

9.1 Dynamic Ablation System (Major Contribution)

A key contribution of this semester is the implementation of a **dynamic ablation system** designed to keep the codebase highly extensible for future research.

Core design:

- **Centralized config** (`ablation_configs.json`) with ON/OFF toggles for each component
- **Instruction templates** (`model_instructions.md`) to modify behavior without changing core code
- **Runtime binding via** `ablation_id` to automatically configure agent behavior
- **Dynamic tool gating** to enable or disable tools per ablation

Impact:

- New ablations can be added without touching core code
- Faster experimentation
- Cleaner research workflow
- Strong reproducibility

This transforms the system into a research-ready experimentation platform.

9.2 Components Tested

- Vector DB
- Crop dictionary
- Progressive filtering
- Confidence scoring
- Web search
- Domain filtering
- Ingestion loop

9.3 Final Ablations

- Baseline
 - Static RAG
 - Static RAG + crop dict
 - Progressive RAG
 - Uncertainty-aware RAG
 - Custom config
 - Full system (no domain filter)
 - Full system (domain filtered)
-

10. Engineering Challenges and Solutions

Challenge 1: Noisy or Missing Metadata

- Solution:
 - Progressive filtering strategy This approach evaluates multiple metadata combinations instead of relying on a single strict filter. By starting with highly specific filters (e.g., location + month + title) and gradually relaxing constraints, the system ensures that relevant results are still retrieved even when some metadata fields are incomplete or incorrect. This significantly reduces retrieval failures caused by missing or inconsistent metadata.
 - Fallback to semantic retrieval When metadata-based filtering does not yield sufficient results, the system falls back to pure semantic similarity search. This ensures that the retrieval process remains

robust and continues to return contextually relevant information based on content meaning rather than structured fields alone.

Challenge 2: Low Retrieval Confidence

- Solution:
 - Adaptive web search + ingestion loop When the confidence score falls below a defined threshold, the system automatically triggers a web search to gather additional information. The retrieved web content is cleaned, chunked, and ingested into the vector database in real time. This allows the system to expand its knowledge base dynamically and re-run retrieval with improved context, leading to higher confidence and more accurate responses.

Challenge 3: Query Ambiguity

- Solution:
 - Crop dictionary-based query enrichment This method enhances ambiguous queries by inserting relevant crop names based on a predefined dictionary. By making implicit context explicit, the system improves retrieval precision and reduces ambiguity. The enrichment process is designed to be safe, with strict validation and fallback mechanisms to ensure that the original query is preserved if no reliable enrichment is found.

Challenge 4: Combinatorial Ablation Complexity

- Solution:
 - Dynamic ablation configuration system The number of possible ON/OFF combinations across retrieval, filtering, confidence, web, and ingestion components grew rapidly as the framework expanded. To manage this configuration space reliably, I implemented centralized ablation definitions and runtime `ablation_id` binding so each experiment can be selected deterministically without manual code edits.
 - Prompt and tool behavior templating Different ablation combinations required different prompt behavior and tool availability. I addressed this by separating instruction templates from code (`model_instructions.md`) and introducing configuration-driven tool gating, which made it possible to map each ablation to the correct prompt/tool policy in a reproducible and maintainable way.

11. Tools and Technologies

- **Vector DB:** Chroma
 - **Embeddings:** SentenceTransformers (BGE)
 - **LLM Serving:** OpenAI-compatible APIs (SGLang / vLLM)
 - **Data Processing:** Python, Transformers, Trafilatura, BeautifulSoup
 - **Parallelism:** Python multiprocessing
 - **Deployment:** Multi-GPU cluster environment
-

12. Key Contributions

- Designed a **metadata-aware RAG architecture**
- Implemented **adaptive retrieval with confidence scoring**
- Built a **dynamic web ingestion loop**

- Developed a **query enrichment mechanism**
 - Engineered a **scalable multi-GPU inference pipeline**
 - Created a **comprehensive ablation framework**
-

13. Future Work

- Transition to the Qdrant vector database to enhance scalability, reliability, and production readiness
 - Conduct comprehensive ablation studies over the summer to generate robust experimental results and insights
 - Optimize system performance to significantly reduce latency per query
-

14. Conclusion

This semester's work resulted in a robust and extensible RAG system capable of:

- Leveraging structured metadata for precise retrieval
- Adapting dynamically to uncertain queries
- Expanding its knowledge base in real time
- Scaling efficiently across multiple GPUs
- Enabling reproducible experimentation through a dynamic ablation framework

The system provides a strong foundation for both research contributions and real-world deployment in domain-specific AI applications.
